

The Gazette

VOL. 1, NO. 008

2026-03-30

COVER STORY

Simple Rust Programming 01 - Checking The Existence of a Directory

class="center">

So I'm at the very early learning stage of Rust, that stage where almost nothing you type in works.

Scott Johnson (fuzzyblog)

lass="center">
C

So I'm at the very early learning stage of Rust, that stage where almost nothing you type in works. And this, for me, usually means "make the learning stick by writing it down". So here goes ...

Note: I wrote this code using the Rust playground , a web based repl, which was invaluable.

id="a-simple-program-for-testing-the-existence-of-a-directory">A Simple Program for Testing the Existence of a Directory

I'm going to start with an assumption that I need to get the location of my input based on the type of the machine: either a Mac or a Linux box. Given that we know that a user's home directory exists in /home on Linux and /Users on Mac, this gives us an easy way to differentiate the two.

id="pass-1-problems-with-scopes">Pass 1: Problems with Scopes

```
fn main () { if path_exists ( "/home" ) { let environment = "unix" ; } else { let environment = "other" ; } println! ( "{", environment ); }
```

```
fn path_exists ( path : & str ) -> bool { if Path :: new ( path ) .is_dir () { return true ; } else { return false ; } }
```

This program has a core problem - the environment variable isn't in scope:

The implication of this is that code within {} braces is a true local block and thus out of scope with what came before.

The Ruby work around for something like this is to have an outer variable that "shadows" the name of the variable within the block and thus allows its value to pass out. Unfortunately Rust identifies that technique as a bug since the outer variable is never read. My refactor on this is to write a function which returns this.

id="pass-2-problems-with-return-types">Pass 2: Problems with Return Types

Here was my first pass at moving environment fetching into a function of its own. I got tired of typing environment so I moved to calling it my_env.

```
fn main () { let my_env = get_env (); println! ( "{", my_env ); }
```

```
fn get_env () { if path_exists ( "/home" ) { return "unix" ; } else { return "other" ; } }
```

```
fn path_exists ( path : & str ) -> bool { if Path :: new ( path ) .is_dir () { return true ; } else { return false ; } }
```

Here are the errors we have:

class="highlight"> Compiling playground v0.0.1 (/playground) error[E0277]: `()` doesn't implement `std::fmt::Display` -> src/main.rs:5:20 | 5 | println!("{}", my_env); | ^^^^^^^ `()` cannot be formatted with the default formatter |

help: the trait ``std::fmt::Display`` is not implemented for ``()`

note: in format strings you may be able to use ``{:?}`` (or `{:#?}` for pretty-print) instead

note: required by ``std::fmt::Display::fmt``

note: this error originates in a macro (in Nightly builds, run with `-Z macro-back-trace` for more info)

```
error[E0308]: mismatched types -> src/main.rs:10:14 | 8 |  
fn get_env() { | - help: try adding a return type: `-> &'static  
str` 9 | if path_exists("/home") { 10 | return "unix"; | ^^^^^^  
expected `()`, found `&str`
```

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0277, E0308.
For more information about an error, try ``rustc -explain E0277``. error: could not compile ``playground``.

To learn more, run the command again with `-verbose`.

id="pass-3-working">Pass 3: Working

The solution to the above was to add a return type or this odd bit of syntax:

Apparently the bit following the `()` identifies the return value of the function as a statically allocated string. This is syntactically similar to the:

but the single quote really threw me.

```
fn main() { let my_env = get_env(); println! ( "{", my_env ); }
```

```
fn get_env () -> &'static str { if path_exists ( "/home" ) { return  
"unix" ; } else { return "other" ; } }
```

```
fn path_exists ( path : & str ) -> bool { if Path :: new  
( path ) .is_dir () { return true ; } else { return false } }
```

id="the-learnings">The Learnings

Here are bullet points for what we learned from writing these 24 lines of code:

The `let` keyword defines a variable.

To insert a variable into string output you use `{ }` inside quotes followed by the name of the variable

If you are going to return something from a function you need to return its type as well

Variables set within `{ }` are local to the `{ }`

{

FEATURES

Ninety Days of Commits

Jake Yesbeck

It has been ninety days since I started the Year of Commits initiative. The criteria behind this challenge was to make a commit to a public Github repository every day. Technically, a commit like the following satisfies that criteria:

However, if I allowed this commit to be my sole contribution for a day, I would be cheating no one except myself. In reality, I have been averaging 3 to 4 decent size commits a day. Spending around an hour or two total per night. Github is an awesome piece of software that tells a visual tale of when I am most productive:

Over this quarter of a year, I have accrued some interesting learnings about developing open source software.

id="writing-software-every-day-just-feels-good">1. Writing software every day just feels good.

Not missing a single day of writing software has made a significant improvement in my confidence and attitude. In the same way exercising daily makes your body feel more sturdy, developing software daily keeps my mind focused and clear. Like warming up a car before a drive, writing software every day makes it very easy for me to jump into a new project or build a new feature. I think most of my fellow software artisans have been in this situation: You go on vacation or maybe just a very relaxing weekend, then the next day at work or when you open that side project again, you end up spending a lot of time getting familiar with the software. Even if you just wrote it a few days ago, the break has made some tiny details less sharp. Writing software every single day helps eliminate this.

id="immature-software-is-expected">2. Immature software is expected

While writing software, there are a total of 0 people that look at what they just wrote and say: "This software is already bad and I have only just written it". However, this statement can be more true than we would like. The potential in a new project is always the best part, right? Developers around the world usually jump at a change to do green field development. After all, with no legacy software around we can make what ever our heart desires. It can be clean, DRY, clever software with all the best design patterns. But, at least in my experience, all software starts off painfully immature. This is just fine. The shape and structure of good software grows over time. This is particularly true for my Year of Commits projects. Since I must write software every single day, I revisit my software extremely frequently. This re-visitation greatly increases the iterations my software goes through and I end up with much more polished work.

id="software-development-is-not-just-making-it-work">3. Software development is not just making it work

If you take away business requirements, deadlines, conversion rates, etc., you are left with something pure: software

development. Year of Commits has enabled me to experience developing software at a slower pace without these extra degrees of "purpose". Because of this, I have learned the difference between software that just works and carefully architect-ed software. I am not boasting that the software I write is now amazingly designed and executed, I have just noticed that when I invest more time, it makes a difference. We, as Software Aristans need time to stop and smell the ones and zeros. Given a decent amount of time and attention, all software can mature to something really extraordinary.

I am optimistic about completing my goal of a Year of Commits. With 3/4 the distance ahead of me, let's hope I can keep that graph solid green.

}

{

A Simple Laravel Hello World

Scott Johnson (fuzzyblog)

As of late I've been working in Laravel and figuring out how Laravel differs from Rails. And while I have a bigger blog post in the works, I thought there was learning value in a simple Hello World implemented in Laravel. Hello World is a simple program that just puts Hello World on the screen.

Here's how I thought a Laravel version of this should work:

Be a whole Laravel application

Be a single controller

Be a single route

Define the text "Hello World" as a variable, \$text, in the controller

Display that on the screen via an HTML template

While this doesn't illustrate models and database connectivity, this actually illustrates how to use a surprising amount of an overall MVC framework.

id="step-1-creating-an-application">Step 1: Creating an Application

The first step is to use composer to generate the overall application with:

The next step is to change into the directory with:

and then open the code base with:

or if you use Sublime:

id="step-2-generating-a-controller">Step 2: Generating a Controller

Back at your command line, you want to generate a controller with:

This will create a file named HelloController in app/http/Controllers:

Inside the controller you want to add the following code (some of which will already be there from the generator):

inside the "class HelloController extends Controller" block of code (it goes inside the {} braces).

id="step-3-adding-a-route">Step 3: Adding a Route

Moving back to the code base, you want to edit the file routes/web.php and add this line:

This adds a single route, /hello, which is then handled by the Hello controller and the hello_index action.

You can test that this route is available by running:

which should show you something like this:

```
class="highlight">+-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
Domain | Method | URI | Name | Action | Middleware | +-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
| GET|HEAD | / | | Closure | web | | GET|HEAD | api/user
| | Closure | api | | | | | auth:api | | GET|HEAD | hello | |
App\Http\Controllers\HelloController@index | web | +-----+
+-----+-----+-----+-----+-----+-----+-----+-----+
```

Note : It is critical that you use the fully name spaced path to the controller (i.e. starting with App). According to what I've read, you can use a use declaration at the top of the routes file to import controllers so you don't have to have the name spacing everywhere but I was unable to ever make that work.

id="step-4-adding-a-variable-to-the-controller">Step 4: Adding a Variable to the Controller

Inside the public function index method of the Hello controller, you want to add / change the following code:

This defines one variable, \$text, which should then be available in the view. The way that it becomes available to the view is by passing a hash into the return statement which has a key of 'text' which is mapped to the variable \$text.

id="step-5-creating-a-view-and-displaying-a-variable">Step 5: Creating a View and Displaying a Variable

Views, in Laravel, live in the directory resources/views so you can create one with the command:

And you then want to edit that file and add this code:

id="step-6-previewing-the-results">Step 6: Previewing the Results

The key command here is:

which starts the application and displays the result on:

At which point you should see "Hello World" come onto the screen.

id="conclusion">Conclusion

This gives a fully functional Hello World application implemented in Laravel. I didn't focus on differences from Rails here in order to simplify this post. In my next post, I'll do more of that.

}

{

Dynamic Languages Strike Back

Steve Yegge

Some guys at Stanford invited me to speak at their EE Computer Systems Colloquium last week. Pretty cool, eh? It was quite an honor. I wound up giving a talk on dynamic languages: the tools, the performance, the history, the religion, everything. It was a lot of fun, and it went over surprisingly well, all things considered.

They've uploaded the video of my talk, but since it's a full hour, I figured I'd transcribe it for those of you who want to just skim it.

This is the first time I've transcribed a talk. It's tricky to decide how faithful to be to my spoken wording and phrasing. I've opted to try to make it very faithful, with only minor smoothing.

Unfortunately I wound up using continuation-passing style for many of my arguments: I'd occasionally get started on some train of thought, get sidetracked, and return to it two or three times in the talk before I finally completed it. However, I've left my rambling as-is, modulo a few editor's notes, additions and corrections in [brackets].

I didn't transcribe Andy's introduction, as it seems immodest to do so. It was funny, though.

Technical corrections are welcome. I'm sure I misspoke, oversimplified, over-generalized and even got a few things flat-out wrong. I think the overall message will survive any technical errors on my part.

The talk...

Thank you everybody! So the sound guys told me that because of a sound glitch in the recording, my normally deep and manly voice, that you can all hear, is going to come through the recording as this sort of whiny, high-pitched geek, but I assure you that's not what I actually sound like.

So I'm going to be talking about dynamic languages. I assume that you're all dynamic language interest... that you've got an interest, because there's a dude down the hall talking about Scala, which is you know, this very strongly typed JVM language (a bunch of you get up and walk over there – exactly.) So you know, presumably all the people who are like really fanatical about strong typing, who would potentially maybe get a little offended about some of the sort of colorful comments I might inadvertently make during this talk – which, by the way, are my own opinions and not Google's – well, we'll assume they're all over there.

All right. I assume you all looked through the slides already, so I don't need to spend a whole lot of time with them. I'll go into major rant-mode here at the end. My goal is... for you guys to come away with, sort of a couple of new pictures in your mind, thinking about how languages have evolved over the last 20 years, where they're going, what we can do to fix them, that kind of thing.

Does anyone here know how to use a Mac? It's showing me this weird, uh... thing... OK. All right. Here goes.

So!

Popular opinion of dynamic languages: sloooooow! They're always talking about how Python is really slow, right? Python is, what, like 10x to 100x slower? And they have bad tools.

And also there's this sort of, kind of difficult-to-refute one, that says at millions of lines of code, they're maintenance nightmares, right? Because they don't have static types. That one, uh, unfortunately we're not going to be able to talk much about, because not many people have millions-of-lines code bases for us to look at – because dynamic languages wind up with small code bases. But I'll talk a little bit about it.

So first of all, one of my compatriots here, who's an actual smart person, like probably everybody in this room, you're all waaay smarter than me – I got invited here for the booger jokes, all right? – he's a languages guy, and he said: "You know, you can't talk about dynamic languages without precisely defining what you mean."

So I'm going to precisely define it. Dynamic languages are, by definition... Perl , Python , Ruby , JavaScript , Lua , Tcl ... all right? [(laughter)] It's the working set of languages that people dismiss today as "dynamic languages." I'll also include Smalltalk , Lisp , Self , Prolog , some of our stars, you know, from the 70s and 80s that, uh, well they'll come up here today too.

I'm deliberately not going down the path of "well, some static languages have dynamic features, and some dynamic languages have static types", because first of all it's this neverending pit of, you know, argument, and second of all, as you're going to see, it's completely irrelevant to my talk. The two... sort of qualities that people associate with "dynamic": one would be sort of... runtime features, starting with eval, and the other would be the lack of type tags, the lack of required type tags, or even just escapes in your type system. These things work together to produce the tools problems and the performance problems, ok? And I'll talk about them, and how they're going to be fixed.

All right!

I just talked about that [slide].

So! Uh... yeah, that's right, I'm at Stanford! Forgot about that. So I've been interviewing for about 20 years, at a whole bunch of companies, and yeah, Stan – every school has this sort of profile, right? You know, the candidates come out with these ideals that their profs have instilled in them. And Stanford has a really interesting one, by and large: that their undergrads and their grad students come out, and they believe that C and C++ are the fabric with which God wove the Universe. OK? And they truly [think]: what is it with all these other languages?

}

{

Validates Type 2.0

Jake Yesbeck

One of the first projects that I worked on during this Year of Commits was `validates_type`. In case that name is too obscure, `validates_type` is a gem for validating that a specific value is exactly the type it is expected to be. The `validates_type` library is compatible to ActiveRecord style validations.

Originally, the `validates_type` gem was very strict with what it validated against. It started with basic included Ruby types like `Integer`, `Float`, and `String`. This was a fine first step but proved too restrictive for basically any use case.

In a recent update, the `validates_type` gem has been extended to validate against any defined type in a system. With this update, the uses of this gem greatly increased. They increased so much that it is possible more than 2 people will use it, and that would be pretty awesome.

id="still-valid">Still Valid

The `validates_type` gem can be useful for a system that cares about exact types at save time. For instance, a model that has an incorrect database column type (because of legacy or other reasons) can still control validation over its data just the same.

If a “junior developer who is now somehow the CTO” created the original system. And if that developer did something like create an `is_published` column on the authors table set as a `varchar`, `validates_type` makes that less of an issue.

```
class="highlighter-rouge"> class Author > author = Author .
new # => # > author . is_published = 'foo' # => "foo" > author .
save! # => ActiveRecord:: RecordInvalid: Validation failed:
is_published is expected to be a Boolean and is not.
```

This way, there is not a bunch of random data in the authors table because of a bad decision made a while ago. More examples on how this gem can be used are found at the prequel to this post and the project’s README.

id="the-new-hotness">The New Hotness

The 2.0 update brought flexibility to `validates_type`. This enables greater control over serialized attributes on ActiveRecord objects.

In a system with an intermediary class inserted between the column assignment and the column serialization, this extension can show its true value.

In an application that deals with Books, each book must store information regarding how it was published:

```
class="highlighter-rouge"> class Book > book = Book .
new > book . publishing_information = { foo: :bar }
> book . save! # => UPDATE "books" SET
```

```
"publishing_information" = $1, "updated_at" = $2
WHERE "books"."id" = $3 [{"publishing_information", "{}"},
["updated_at", "2015-11-30 05:04:09.480707"], ["id", 3]]
```

If, like the above example, the `PublishingInformation` class is forgiving, this will silently fail. The `book.publishing_information` will not be set to the correct value.

On the other hand, if the `PublishingInformation` does fail, it might not do so in an obvious way.

However, with `validates_type`, this issue becomes explicit.

Adding the type validation to the book model:

```
class="highlighter-rouge"> class Book > b = Book .
new > b . publishing_information = { a: :b } > b .
save! # => ActiveRecord:: RecordInvalid: Validation failed:
publishing_information is expected to be a PublishingIn-
formation and is not.
```

Now, a readable error is produced and nothing is left to the imagination.

id="a-reasonable-start">A Reasonable Start

Was the used example extremely specific? Yes. But, I would wager that at least one application out there has had a problem similar to this and `validates_type` can ensure that it never happens again. Hopefully, this example and the problem it did end up solving with at least act as inspiration for this library’s other uses out in the wild.

I have found a use for this type of validation and hope others can as well. If a use case arises that this gem does not support but could easily, I invite everyone to contribute to it.

}

{

Identifying vulnerabilities in GitHub Actions & AWS OIDC Configurations

Tinder · Tinder Tech

Authored by: Rojan Rijal, Tinder Security Labs | Johnny Nipper, Sr. Director | Tanner Emek, Sr Engineering Manager

In 2021, GitHub released support for OpenID Connect (OIDC) for GitHub Actions (GHA), allowing developers to securely interact with their infrastructure resources in Amazon Web Services (AWS), and other major cloud service providers. The OIDC support allows GHA jobs to retrieve short-lived session tokens on-demand rather than using the private key and credential files as secrets. At Tinder Security Labs, we identified that certain configurations when setting up OIDC with GHA could result in vulnerabilities allowing external attackers to get access to vulnerable organizations' cloud infrastructure. In addition, we have written and published a black-box assessment tool that monitors and flags when vulnerable configurations are noticed for AWS environments. Through this blog, we will share examples of vulnerable configuration, case-studies with external organizations, mitigation examples, and abuse identification techniques.

Inner workings of AWS OIDC + GitHub

One of the cloud providers that supports using OpenID Connect with GHA is AWS. In order to configure OIDC in AWS, two things are needed: an IAM role and an Identity provider that can be linked to the IAM role. From GitHub end, no configuration is needed and the setup is done in the GHA workflow.

GitHub setup

Using OIDC through a GitHub workflow is straightforward and does not require any technical setup on GitHub. The only requirement is to have id-token permissions for a workflow/job set to write permissions. This is needed for the job to obtain and store a JWT ID token from GitHub's OIDC provider. In order to do so, a request is made by the job to a `https://pipelines.actions.githubusercontent.com` endpoint as specified by `ACTIONS_ID_TOKEN_REQUEST_URL` environment value. This request requires a job JWT, which is stored in `ACTIONS_ID_TOKEN_REQUEST_TOKEN` environment value.

The response from sending a request to request URL with the request token will return a JWT ID Token. This is then sent to the OIDC supporting cloud provider such as AWS to exchange for a short-term session token. This JWT is formatted with couple of required pieces of information:

kid & x5t and alg in the header — These values can be used by cloud providers to make sure the JWT is valid and signed correctly.

sub & aud value in the payload — The aud value is the audience value specified in the workflow. By default, this will be the repository URL or in cases of actions like

`configure-aws-credentials` `sts.amazonaws.com`. This value is customizable. sub value is the subject claims for the JWT. This by default contains the repository namespace and branch reference. A format of sub can be `repo:org-Name/repo_name:ref:branch_ref`

AWS setup

When creating an identity provider a client ID and identity provider URL is needed. This works as an initial authentication to make sure that the request originated for an allowed client. With GitHub, the identity provider is linked to `https://token.actions.githubusercontent.com`. This URL contains the information about the OpenID configuration on GitHub end. AWS uses this to make sure the originating request is valid and allowed. The audience value by default is the URL for the GitHub repository that is trying to assume the role for example `https://github.com/TinderSec/sample_repo`. However, it is also customizable. For example, `configure-aws-credentials` actions provided by AWS uses `sts.amazonaws.com` as a default audience value. As a result, a sample identity provider configuration may look like this:

Once the identity provider has been created, multiple IAM roles can be assigned to it. These roles designate what resources and permissions that a GHA job can perform in the AWS account. In addition, a trust relationship can be set up in the role to validate the request sent by GitHub to request the short-lived session token for the role. The trust relationship can be configured to validate two specific keys: aud and sub . These are the values sent by GitHub within the JWT for the job when requesting for the short-lived credentials. The aud value is the audience string specified in the workflow (`sts.amazonaws.com` by default for `configure-aws-credentials` or the repository url for custom usages). The sub value is the subject payload in the request sent by GitHub. By default, this contains the repository namespace and the branch for example, `repo:org_name/repo_name:ref:branch_ref` .

A sample trust policy to create a secure OIDC that does a direct string match for audience and a wildcard match for the subject will look like this:

This setting makes sure the request comes with an audience set to `sts.amazonaws.com`, and the repository is locked to any repo beginning with `repo_name` in `org_name` organization.

Misconfigurations in roles

Our analysis showed a common occurrence where a trust policy for the IAM role was misconfigured. As a result, external repositories not associated with the organizations were able to assume the AWS roles against organizations and access the configured resources. The most common occurrence we noticed was a missing a check for the subject claim with a trust policy similar to this:

Initially, we were not sure how many roles would be mis-

}

{

The AI boom has plunged a small Pennsylvania town into chaos

Rebecca Egan McCarthy · Grist

class="has-drop-cap has-default-font-family">"I don't like to see anyone upset," said Nick Farris of Provident Real Estate Advisors. He was sitting in the front of a crowd of roughly 150 inside Valley View High School's auditorium in Archbald, a town of about 7,500, huddled between two mountain ranges in Pennsylvania's Lackawanna Valley. Farris was there to represent the developer for Project Scott, one of many data center campuses coming to town. "However," he said. "I think that this is the best data center site in this area of the country, by far." The audience had been fairly quiet, bundled in thick coats against the late January cold. But as Farris spoke about data centers as a boon for communities, they began to laugh, drawing a rebuke from town officials.

class="has-default-font-family">"What about the children?" someone shouted from the crowd. The children were watching from the walls; long banners of Valley View Performing Arts students hanging around the auditorium like championship pennants. Project Scott and four other data facilities will sit just a few thousand feet from the middle and high schools.

class="has-default-font-family">"Isn't there a missile plant next door?" Farris said, getting aggravated. He was referring to Lockheed Martin's 350,000-square-foot Missiles and Fire Control facility directly next to the high school, parts of which are highly contaminated.

class="has-default-font-family">"That sucks too!" another attendee yelled back. This was nothing to worry about, Farris tried to convince the audience. This would bring in tax revenue, he said. It was just an office park, albeit one with roughly 450 diesel backup generators.

class="has-default-font-family">"It's going to be away from everyone," Farris kept repeating, to rising jeers. He was wearing a knit turtleneck with a large American flag emblazoned across the chest. "It's not going to bother anyone."

class="has-default-font-family">The specifications say something different: Five developers are planning to build six data center campuses in Archbald, which will cover a full 14 percent of the town, evict a trailer park, and border many residential properties. One campus alone, as The Scranton Times-Tribune reporter Frank Lefneskey pointed out, is expected to use more power than the region's largest power plant is able to produce.

class="has-default-font-family">Pennsylvania has become an epicenter of the data center boom in the United States, with over 50 campuses in development. Eleven of them are slated for Lackawanna County alone. Archbald, with six campuses composed of 51 massive buildings, has the most of any municipality in Pennsylvania.

class="has-default-font-family">Despite the public outcry, it has been surprisingly difficult to learn what Archbald's

elected officials think of the massive industry moving in. None of the town's seven council members responded to my emails, so I stopped by the borough administration building in person a few weeks before Christmas, and was told to wait in the lobby while they held an informal closed meeting in council chambers. When the door opened, it was clear that anyone who actually had the power to make or break the data center plans had quietly filed out the back, leaving me with Archbald's unelected borough manager, Dan Markey, who essentially runs the town, but cannot vote for or against development.

class="has-default-font-family">What did he think about artificial general intelligence, or AGI — the idea that eventually these efforts will produce something like a "computer god" capable of solving climate change, ending hunger, revealing the full breadth of science, and performing any number of other miracles? "I believe in one God, and it's not a computer," Markey told me evenly. By now, hundreds of towns across the country have been caught up in the rush to build large language models that can parse unfathomable amounts of data to write copy, answer any query, develop new vaccines, and likely render a large number of jobs obsolete. That rush accelerated into an all-out arms race over the past year. What it means in practice is an enormous amount of data centers — enough to approximate a minor deity and enough, as OpenAI co-founder and former chief scientist Ilya Sutskever once speculated, to "cover the Earth." Markey told me he had ChatGPT on his phone, but didn't use it much.

class="has-default-font-family">"I don't think anyone in their right mind wants to see the world covered in data centers," Markey said. "[But], according to Pennsylvania law, we have to have a zone for everything. An adult bookstore, a strip club, a concrete and asphalt plant — anything that wants to come here. We have to have a zone for it. If it's not zoned, it's allowed to go anywhere."

class="has-default-font-family">In most states, towns have the right to exclude businesses that they find disagreeable — a wealthy suburb, for example, would likely reject a landfill or gas plant — but in Pennsylvania, towns must allocate some patch of land to these "undesirable industries." Some municipalities deal with this by forming what are called zoning collaboratives, which allow them to plan as a region for pollution. One town might get data centers, another gas plants, another a landfill. Markey approached the nearby boroughs of Blakely and Dickson City to discuss the possibility, but the data center development rush has outpaced him.

class="has-default-font-family">Whatever's attracting data centers to the area, it's forced Markey to answer for a rush of development, unprecedented since the town was settled in the 1840s in service of an industry that will bring vanishingly few jobs. Residents kept bringing up the threat of collapse — the network of empty mine shafts running underground, the town beneath the town, the structural instability that would accompany these massive buildings. The data centers would drive homeowners into town, they said

}

{

When Jupyter Notebooks Won't Import Libraries

Scott Johnson (*fuzzyblog*)

I don't claim to be an expert when it comes to Python. At best I'm an apprentice striving to be a journey man. One of the interesting tools in the Python / Data Science ecosystem is the Jupyter Notebook which gives a cell based representation of code, visualizations, documentation and execution flow and allows you to package things up for distribution i.e. hand your work, in a complete fashion, from say Data Scientist 1 ("Rebekah") to Data Scientist 2 ("Dawn"). This is a laudable goal and one that I theoretically agree with.

Note : Jupyter Notebooks do NOT include data so that's still external to the notebook, something that can easily bite you (as it is currently biting me).

I know a lot of pythonistas simply install libraries to their local machine and just have a collection of tools that they throw at problems. This, however, is a terrible practice due to code deprecation, version conflicts, etc. I say this with authority because I've been through this in the Ruby world before we all regularly started using Ruby virtual environments / Ruby version managers like RBENV / RVM which manage dependencies on a per project basis. Knowing this, my first practice with Python, is to always create a virtual environment, generally using Virtual ENV (VENV).

So, when I set out today to use a Jupyter Notebook, my first approach was to make this work with a Jupyter Notebook. And, alas, I haven't been so lucky as to make this work cleanly with a Python virtual environment like VENV. Here are some of the things I tried:

Installing jupyter into the virtual env and running from bin/jupyter notebook

Changing the Kernel

If you're curious about how to use Python Virtual Environments, I wrote a solid tutorial back in September that I used to get a full installation of Tensor Flow up and running. I've referred back to this over and over, each time I needed a Python Virtual Environment, so I know it works.

I'm sure there is a way to mess about with virtual environments and Jupyter Notebooks to make them work but, honestly, I'm skeptical on notebooks and how they obfuscate code and data together anyway so I thought "How do I just make this a Python script". I took this approach because I was absolutely certain that I could make a virtual environment work with just Python. And, thanks to my pairing partner, Grant, there is, indeed, a way.

id="making-a-jupyter-notebook-into-a-python-script-with-a-virtual-env">Making a Jupyter Notebook into a Python Script with a Virtual Env

Follow the instructions here to setup VENV for a new project in a new directory.

Use File menu, Export as Python to write a single Python script representing the notebook.

Create a requirements.txt file as per the instructions here .

Go through the Python code that was exported and convert the import / from instructions to the right PyPy package index name entry in the requirements.txt file. Be aware that there isn't a straight correspondence between the import statements and the package names. For example you import from "metal" in the notebook I'm porting but the package name is actually "snorkel-metal" and you import from sklearn but the package name is actually scikit-learn. Python's there's only one way to do this mantra, in the area of package management, is just plain crap. Sigh.

Run `pip3 install -r requirements.txt`

Run your python script and then adjust the requirements.txt file accordingly. You will almost certainly need to change some things but, by and large, I'm finding that this process does work.

}

{

Ask a Climate Therapist: How can I balance my travel itch with guilt about emissions?

Leslie Davenport · Grist

Almost all of the money I save goes toward travel — meeting friends abroad, visiting relatives, or checking national and international parks off my bucket list. I try to be as sustainable in my personal life as possible, but I love traveling, despite knowing that travel is where some of our greatest carbon emissions come from. Every time I fly now, I feel this guilt and fear, and it's sometimes hard to balance it with my excitement. How would you recommend proceeding when some of the decisions we make in our personal lives are at odds with our beliefs and goals?

The tension you're describing — the pull between loving this world and worrying about harming it — is a sign of a very intact compassion compass. Sometimes, that compassionate awareness stretches us beyond what's comfortable or easy to resolve.

Don't dismiss your discomfort, because it's inviting you to stay awake to your impacts on the planet — but don't let it shut down your capacity for joy. If guilt grows so heavy that it diminishes your ability to connect and feel alive, it's lost any usefulness it once offered. The work now is to metabolize your guilt and fear into something that guides you rather than pulls you apart.

Ask a Climate Therapist tackles your questions about how to navigate the emotional side of climate change, with leading climate-aware therapist Leslie Davenport. Have a question? Ask it here !

That opens up more nuanced choices than simply "Go or don't go?" Maybe you'll travel less often but stay longer, explore more locally using lower-carbon methods of transport, or prioritize trips that deepen relationships.

You might also focus on destinations where you can make a positive impact as a tourist. In countries like Costa Rica, Rwanda, Tanzania, and Bhutan, responsible tourism helps keep conservation efforts alive by funding wildlife protection, supporting local communities, and sustaining fragile ecosystems.

There's a larger truth in this as well. You're just one person — and your choices matter, but they were never meant to bear the full weight of a systemic crisis. You didn't build the fossil fuel-dependent systems that make flying one of the only practical ways to stay connected across long distances or encounter the wider world. Guilt can keep us preoccupied with ourselves rather than on the larger structures driving the harm.

Continue to balance your personal choices with collective ones. To start, you might want to discuss your dilemma with friends and family — whether they're your travel companions, or people in your life who may have already made different calculations

about travel. (In last month's column, I shared similar advice with a question asker who could well be on the other side of that conversation .) Having a shared understanding with loved ones might help you process your inner tension, while also letting your care for the planet ripple out in community.

Take some time to reflect, and then settle into commitments that feel right for you. Return to your decision from time to time and make changes as needed. When you feel a pang of guilt, let it keep you honest but not overwhelmed.

Remember: The places you visit, the people you love across long distances, and the wonder you feel in national parks aren't separate from your love for the Earth. They're expressions of it. Add in a dose of reflection, gratitude, and perhaps even awe, and your journeys will strengthen the passion and resilience that allow you to stay engaged in your work. In this way, the same travel that stirs your guilt can also help protect what you love. It's not a clean or simple solution, but it's real.

The aim is to stay present for the whole of it — to allow your love for our planet to be large enough to hold grief, responsibility, and joy at the same time. Let yourself embrace and enjoy the world wholeheartedly, even as you engage in protecting it.

I'm Leslie Davenport , a licensed therapist, educator, speaker, consultant, and internationally recognized voice on the emotional and psychological dimensions of climate change. If you've got a question about climate and mental health, please submit it here for a future column .

}

{

The 12 Bugs of Christmas - A Software Developers' Version

Scott Johnson (*fuzzyblog*)

This isn't mine but it was forwarded to me via a group chat and, in it, I found a deep, profound and abiding truth:

id="1—for-the-first-bug-of-christmas-my-manager-said-to-me">1. For the first bug of Christmas, my manager said to me:

See if they can do it again.

id="2—for-the-second-bug-of-christmas-my-manager-said-to-me">2. For the second bug of Christmas, my manager said to me:

Ask them how they did it and

See if they can do it again.

id="3—for-the-third-bug-of-christmas-my-manager-said-to-me">3. For the third bug of Christmas, my manager said to me:

Try to reproduce it

Ask them how they did it and

See if they can do it again.

id="4—for-the-fourth-bug-of-christmas-my-manager-said-to-me">4. For the fourth bug of Christmas, my manager said to me:

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

id="5—for-the-fifth-bug-of-christmas-my-manager-said-to-me">5. For the fifth bug of Christmas, my manager said to me:

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

id="6—for-the-sixth-bug-of-christmas-my-manager-said-to-me">6. For the sixth bug of Christmas, my manager said to me:

Reinstall the software

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

id="7—for-the-seventh-bug-of-christmas-my-manager-said-to-me">7. For the seventh bug of Christmas, my manager said to me:

Say they need an upgrade

Reinstall the software

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

id="8—for-the-eighth-bug-of-christmas-my-manager-said-to-me">8. For the eighth bug of Christmas, my manager said to me:

Find a way around it

Say they need an upgrade

Reinstall the software

Ask for a dump

Run with the debugger

Try to reproduce it

Ask them how they did it and

See if they can do it again.

id="9—for-the-ninth-bug-of-christmas-my-manager-said-to-me">9. For the ninth bug of Christmas, my manager said to me:

Blame it on the hardware

Find a way around it

Say they need an upgrade

Reinstall the software

Ask for a dump

Run with the debugger

Try to reproduce it

For the fifth bug of Christmas, my manager said to me: Ask for a dump Run with the debugger Try to reproduce it Ask them how they did it and See if they can do it again.

Scott Johnson (fuzzyblog)

}

{

Running Multiple Rails Apps Concurrently with Foreman and Procfile.dev

Scott Johnson (fuzzyblog)

As I've said, I build a lot of side projects and I really, really like the model of having:

ALL MY APPS RUNNING CONCURRENTLY

I may be a scattered, distracted developer trying to do too damn much but that's my damn right. And I have 64 gigs of RAM so why shouldn't I be this way.

What I want is to be able to switch from app to app and make changes. This is important because some apps provide APIs which other apps rely on and having to figure out what thing is on what port, etc, is just plain distracting.

Foreman and Procfile.dev is a way around this but there's a major hitch in your getalong (as my Texas wife might say).

Here's a sample Procfile.dev for an app I'm building called Cartazzi which makes a developer's life easier:

And here's a Profile.dev for another application called Poolwizard which helps you maintain your swimming pool:

If you run Cartazzi and Poolwizard together then you're going to get crashes and here's the error:

09:11:53 web.1 | started with pid 72877

09:11:53 css.1 | started with pid 72878

09:11:53 js.1 | started with pid 72879

09:11:53 worker.1 | started with pid 72881

09:11:53 js.1 | yarn run v1.22.5

09:11:53 css.1 | yarn run v1.22.5

09:11:53 css.1 | \$ tailwindcss -postcss -i ./app/assets/stylesheets/application.tailwind.css -o ./app/assets/builds/application.css -watch

09:11:53 js.1 | \$ node esbuild.config.js -reload

09:11:54 js.1 | node:events:371

09:11:54 js.1 | throw er; // Unhandled 'error' event

09:11:54 js.1 | ^

09:11:54 js.1 |

09:11:54 js.1 | Error: listen EADDRINUSE: address already in use :::5200

09:11:54 js.1 | at Server.setupListenHandle [as _listen2] (node:net:1306:16)

09:11:54 js.1 | at listenInCluster (node:net:1354:12)

09:11:54 js.1 | at Server.listen (node:net:1441:7)

09:11:54 js.1 | at buildAndReload (/Users/sjohnson/Sync/coding/rails/poolwizard/esbuild.config.js:54:6)

09:11:54 js.1 | at Object. (/Users/sjohnson/Sync/coding/rails/poolwizard/esbuild.config.js:100:3)

09:11:54 js.1 | at Module._compile (node:internal/modules/cjs/loader:1109:14)

09:11:54 js.1 | at Object.Module._extensions..js (node:internal/modules/cjs/loader:1138:10)

09:11:54 js.1 | at Module.load (node:internal/modules/cjs/loader:989:32)

09:11:54 js.1 | at Function.Module._load (node:internal/modules/cjs/loader:829:14)

09:11:54 js.1 | at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:76:12)

09:11:54 js.1 | Emitted 'error' event on Server instance at:

09:11:54 js.1 | at emitErrorNT (node:net:1333:8)

09:11:54 js.1 | at processTicksAndRejections (node:internal/process/task_queues:83:21) {

09:11:54 js.1 | code: 'EADDRINUSE',

09:11:54 js.1 | errno: -48,

09:11:54 js.1 | syscall: 'listen',

09:11:54 js.1 | address: '::',

09:11:54 js.1 | port: 5200

09:11:54 js.1 | }

09:11:54 js.1 | error Command failed with exit code 1.

09:11:54 js.1 | info Visit <https://yarnpkg.com/en/docs/cli/run> for documentation about this command.

09:11:54 js.1 | exited with code 1

09:11:54 system | sending SIGTERM to all processes

09:11:54 worker.1 | terminated by SIGTERM

09:11:54 css.1 | exited with code 1

09:11:54 web.1 | terminated by SIGTERM

The first time I looked at this, I saw the port error and I assumed that the error was the rails server because I knew that was running on a specific port. The actual error though here is `node` – which ISN'T EVEN IN THE Procfile.dev because it is run behind the scenes by yarn.

```
}
```

standard-article(title: [New I/O flavors in ePouta], author: [Jukka Nousiainen], source-name: [CSC Cloud Team Blog], images: (), paragraphs: ([style="text-align: left;">], [About a year ago we released the I/O Virtual Machine flavors in cPouta. As of July, similar flavors are now also available in ePouta; you can find an exhaustive list here .], [The I/O flavors are provided with the same disclaimer as with their cPouta counterparts: both the root disk and the ephemeral disk for data storage are hosted on a striped (RAID0) array of SSD disks. That means you get a lot of I/O, but absolutely no guarantees for the persistence or health of VMs in case a node or a disk breaks.], [Thus these flavors are most natural to use with software which clusters nicely, such as Hadoop or Cassandra. To further ensure each VM in a cluster ends up on separate nodes, we recommend creating an anti-affinity server group .],), insert-map: (:), word-count: 140, edited-for-length: false, debug-mode: false,)

standard-article(title: [Turning Off Ruby Deprecation Warnings When Running Tests], author: [Scott Johnson (fuzzyblog)], source-name: [Scott Johnson (fuzzyblog)], images: (), paragraphs: ([The combination of Rails 6 and Ruby 2.7, or perhaps just Rails 6, introduced some new deprecation warnings and, lately, I've been seeing this cruft constantly:], [Running via Spring preloader in process 34024], [/Users/sjohnson/.rvm/gems/ruby-2.7.0/gems/activemodel-6.0.2.1/lib/active_model/type/integer.rb:13: warning: Using the last argument as keyword parameters is deprecated; maybe ** should be added to the call], [/Users/sjohnson/.rvm/gems/ruby-2.7.0/gems/activemodel-6.0.2.1/lib/active_model/type/value.rb:8: warning: The called method `initialize' is defined here], [/Users/sjohnson/.rvm/gems/ruby-2.7.0/gems/activerecord-6.0.2.1/lib/active_record/connection_adapters/postgresql/oid/specialized_string.rb:12: warning: Using the last argument as keyword parameters is deprecated; maybe ** should be added to the call], [/Users/sjohnson/.rvm/gems/ruby-2.7.0/gems/activemodel-6.0.2.1/lib/active_model/type/value.rb:8: warning: The called method `initialize' is defined here], [And, in the immortal words of Twisted Sister , we're not going to take it anymore.], [The solution is a bit more byzantine than I have found in the Rails world and a bunch of the options in the Stack Overflow "top of Google" but no longer accurate post no longer work or aren't quite what you expect.], [The easy solution is to add:], [to the top of config/environments/test.rb so it looks like this:], [# The test environment is used exclusively to run your application's # test suite. You never need to work with it otherwise. Remember that # your test database is "scratch space" for the test suite and is wiped # and recreated between test runs. Don't rely on the data there! config . cache_classes = false], [Obviously the rest of the file has to be there but that \$VERBOSE=nil silences all the cruft.], [One other solution, RUBYOPT="-W0", prepended before you run the test doesn't work with the new use of "rails" to run your tests but does work with "rake" to run your tests

brief-group((brief-item([Scott Johnson (fuzzyblog)], source-name: [Scott Johnson (fuzzyblog)], [This falls into the category of "I'm old and can't remember this so must, must, must write it down" because I keep losing this html page . If you need to change the text on a submit button for a Rails form implemented with simple_form then use:

```
class="language-plaintext highlighter-rouge">
```

```
class="highlight">))
```

```
)
```

```
so:], [Now, that said, $VERBOSE=nil is
a better option because it goes in at
the project level and affects all devel-
opers on the project.], ), insert-map:
(:), word-count: 331, edited-for-length:
false, debug-mode: false, )
```

The easy solution is to add: `class="language-plaintext highlighter-rouge">class="highlight">` \$VERBOSE=nil to the top of `config/environments/test`.

Scott Johnson (fuzzyblog)

```
standard-article( title: [Latent Effects for Reusable Lan-
guage Components], author: [Lambda the Ultimate],
source-name: [Lambda the Ultimate], images: (), para-
graphs: ( [The development of programming languages can
be quite complicated and costly. Hence, much effort has
been devoted to the modular definition of language features
that can be reused in various combinations to define new
languages and experiment with their semantics. A notable
outcome of these efforts is the algebra-based "datatypes
"a la carte" (DTC) approach. When combined with alge-
braic effects, DTC can model a wide range of common
language features. Unfortunately, the], [current state of the
art does not cover modular definitions of advanced control-
flow mechanisms that defer execution to an appropriate
point, such as call-by-name and call-by-need evaluation, as
well as (multi-)staging. This paper defines latent effects, a
generic class of such control-flow mechanisms. We demon-
strate how function abstractions, lazy computations and
a MetaML-like staging can all be expressed in a modular
fashion using latent effects, and how they can be combined
in various ways to obtain complex semantics. We provide
a full Haskell implementation of our effects and handlers
with a range of examples.], [Looks like a nice generalization
of the basic approach taken by algebraic effects to more
subtle contexts. Algebraic effects have been discussed here
on LtU many times . I think this description from section
2.3 is a pretty good overview of their approach:], [LE&H
is based on a different, more sophisticated structure than
AE&H's free monad. This structure supports non-atomic
operations (e.g., function abstraction, thunking, quoting)
that contain or delimit computations whose execution may
be deferred. Also, the layered handling is different. The idea
is still the same, to replace bit by bit the structure of the
tree by its meaning. Yet, while AE&H grows the meaning
around the shrinking tree, LE&H grows little "pockets of
meaning" around the individual nodes remaining in the
tree, and not just around the root. The latter supports
deferred effects because later handlers can still re-arrange
the semantic pockets created by earlier handlers.], ), insert-
map: (:), word-count: 333, edited-for-length: false, debug-
mode: false, )
```

```
standard-article( title: [A nod to checklists], author: [Eddie
Smith (Practically Efficient)], source-name: [Eddie Smith
(Practically Efficient)], images: (), paragraphs: ( [Gabe
whipped up a great list of checklist tools. My favorite aspect
of his post is that there's no clear winner. There shouldn't
be.], [Checklists can come in all forms, and the ideal format
depends entirely on the application. For me, checklists
```

```
standard-article( title: [Coding with Python Dictionaries the
Ruby Way With Respect to Missing Keys], author: [Scott
Johnson (fuzzyblog)], source-name: [Scott Johnson (fuzzy-
blog)], images: (), paragraphs: ( [I recently had an interest-
ing experience with Python dictionaries that made question
one of Python's foundational principles i.e. TOOWTDI:],
[There should be one- and preferably only one -obvious
way to do it. Although that way may not be obvious at first
unless you're Dutch. More...], [The application was a large
scale JSON processor and, in Python, JSONs are mapped
to Python dictionaries which correspond to Ruby hashes.
The issue is how you handle keys that don't exist. Here's
the Ruby way:], [class="highlight"> irb ( main ): 001 : 0 >
hash = {} {} irb ( main ): 002 : 0 > hash [ "foo" ] = "bar"
"bar" irb ( main ): 003 : 0 > hash [ "blah" ] nil, [And here's
the Python way:], [class="highlight"> >>> dict = {} >>> dict
[ "foo" ] = "bar" >>> dict [ "blah" ] Traceback ( most recent
call last ): File " ", line 1 , in KeyError : 'blah' >>>], [At the
heart of things is whether or not a non-existent key causes
an exception to be thrown. I look at this from the cultural
perspective of the language's creators.], [Ruby says:], ["Oh
hell no; good heavens why would a non existent key cause
an error. We're Japanese at our heart and tossing an excep-
tion for this feels, well, ahem, rude."], [While Python says:],
["Absolutely a missing key must translate to an exception;
we are Dutch after all!"], [The interesting thing here is that
despite the foundational principle of TOOWTDI, Python
offers a way to emulate Ruby's default behavior - the .get()
syntax. Here's an example:], [class="highlight"> >>> dict =
{} >>> dict [ "foo" ] = "bar" >>> dict . get ( "foo" ) 'bar' >>>
dict . get ( "blah" ) >>> result = dict . get ( "blah" ) >>> print
( result ) None], [My vastly more Pythonic co-worker, Grant,
gave me this explanation when I raised this issue in our
team Slack:], [@sjohnson You can make Python dicts return
None if the key doesn't exist by using the .get class function
instead of the square-brackets key notation. So if d is your
dictionary, you can use d.get(key) instead of d[key]. The
latter will throw a KeyError if the key doesn't exist, whereas
the former will just return a null value.], [Thanks Grant!], ),
insert-map: (:), word-count: 397, edited-for-length: false,
debug-mode: false, )
```

```
standard-article( title: [A Regular Expression for Validating
An Internet Domain], author: [Scott Johnson (fuzzyblog)],
source-name: [Scott Johnson (fuzzyblog)], images: (), para-
graphs: ( [Despite the power and truth of Jamie Zawinski's
law:], [Regular Expressions: Now You Have Two Problems
Jeff Atwood's Perspective], [Like Jeff, I too really, really love
regular expressions or regexes. I use this one a lot and I
```


make sense when I need to see not only what needs to be done, but also what I've already done . Apps that automatically “vanish” completed tasks fail to do the latter.], [For me, sometimes there's just no substitute for a spreadsheet for large checklists, especially if each item can have multiple statuses or dimensions. Sometimes adding more columns is way more efficient than adding more tasks (rows).], [For packing lists, I've tried so many apps, but OmniOutliner is the best for me. It's simple checkbox feature is perfect, and I have several templates I use for different types of trips.], [Sometimes an Apple Note will suffice, and sometimes I just “x” lines in Drafts for a quick and dirty grocery list.], [When I'm working with large numbers of LaTeX files on my Mac, I use file colors, prefix schemes, and even moving files from one folder to another to keep what I've processed and what I haven't.], [Checklists are as old as civilization and are one of the most fundamental ways to augment the human mind, which needs help seeing where it's been and where it needs to go. Everyone can benefit from checklists. Just check out The Checklist Manifesto .],), insert-map: (:), word-count: 244, edited-for-length: false, debug-mode: false,)

finally learned to use \S (Any non-whitespace character) so here's a regex], [that I wrote yesterday to “validate” the permitted characters in an Internet domain. I was all proud of this and wrote this blog post only to realize that pride really does goeth before a fall – this will NOT correctly validate an Internet domain. As I write this post, I realize that the number of allowed characters in an Internet domain are actually NOT any non-whitespace characters and here's the proof that I actually got that wrong yesterday when I put something online using it:], [Note : The fact that Rubular allows through an _ which is NOT a valid character in domains is problematic.], [So the right way to do this, DAMN IT, is something like this:], [And this actually works:], [The [A-Za-z0-9-]+ is a “character class” which says “Any uppercase or lowercase letter plus 0-9 plus a -” are allowed (any order, any quantity)“.], [Regular Expressions – Now you have two problems.],), insert-map: (:), word-count: 214, edited-for-length: false, debug-mode: false,)

For me, sometimes there's just no substitute for a spreadsheet for large checklists, especially if each item can have multiple statuses or dimensions.

Eddie Smith (Practically Efficient)

The Gazette

Vol. 1, No. 008 · 2026-03-30

Curated and composed automatically.